

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Session 21

Service Mesh & mTLS — Istio

Sidecar Proxy Pattern • Mutual TLS • Traffic Management • Mesh-Level Resilience

Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

Session 21

Service Mesh & mTLS — Istio

Sidecar Proxy Pattern • Mutual TLS • Traffic Management • Mesh-Level Resilience

1. Learning Objectives

- Explain what a service mesh and sidecar proxy are.
- Distinguish the Istio Data Plane (Envoy) from the Control Plane (Istiod).
- Explain how sidecars intercept service-to-service traffic without changing Spring Boot code.
- Explain TLS versus mutual TLS (mTLS).
- Enable automatic sidecar injection and enforce STRICT mTLS.
- Explain how mTLS complements OAuth2/JWT rather than replacing it.
- Configure VirtualService and DestinationRule for weighted traffic routing.
- Configure outlier detection as mesh-level resilience.
- Compare Resilience4j with Istio and make an architectural choice.

Part 1 — Why Do We Need a Service Mesh?

2. The Problem We Have Today

As a microservices platform grows, every service needs networking capabilities such as TLS, retries, timeouts, circuit breaking, telemetry, routing, and security. Implementing all of these independently inside every service creates duplication.

```
Order Service
  |
  | HTTP
  v
Inventory Service
```

A service mesh moves many infrastructure-level networking concerns outside the application.

3. Cross-Cutting Concerns

```
Product Service
+-- Resilience4j
+-- OAuth2
+-- Micrometer
+-- Retry / Timeout logic

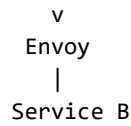
Order Service
+-- Resilience4j
+-- OAuth2
+-- Micrometer
+-- Retry / Timeout logic
```

A mesh can apply common networking policies even when services are written in different languages.

4. What Is a Service Mesh?

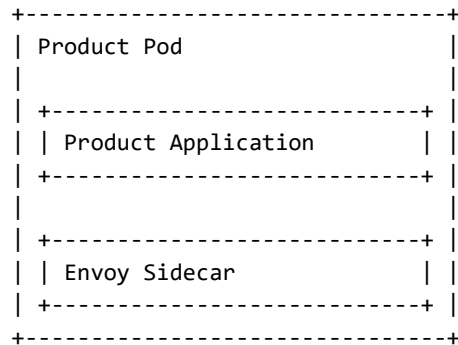
A service mesh is an infrastructure layer dedicated to managing communication between services. Istio implements this primarily through proxies deployed alongside workloads.

```
Service A
  |
  Envoy
  |
  | mesh traffic
```

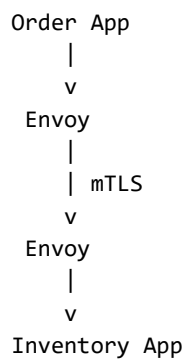


5. Sidecar Proxy Pattern

A sidecar is a second container deployed in the same Kubernetes Pod as the application container.



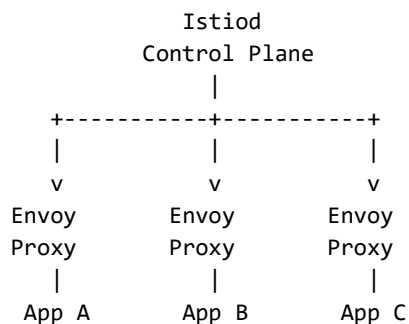
6. How a Request Travels



The application can continue using its normal HTTP client. Envoy handles the mesh-level networking behavior.

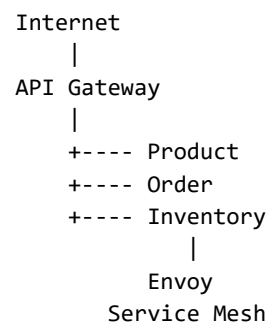
7. Data Plane vs Control Plane

The Data Plane is made of Envoy proxies. They handle actual network traffic. The Control Plane is Istiod, which distributes mesh configuration and policies to the proxies.



8. Service Mesh vs API Gateway

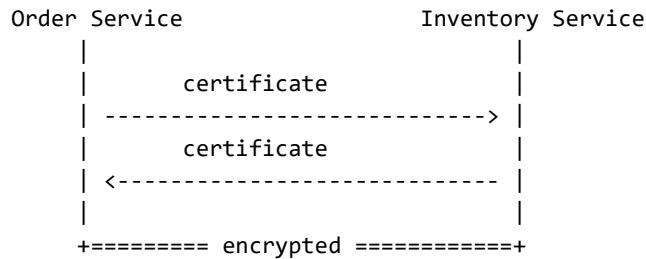
An API Gateway primarily manages traffic entering or leaving the platform. A service mesh primarily manages service-to-service, or east-west, traffic.



Part 2 — Mutual TLS

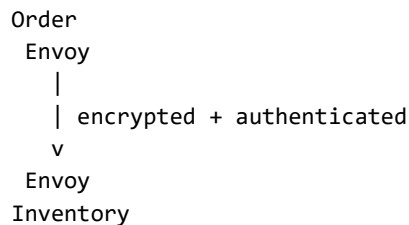
9. TLS vs mTLS

With ordinary HTTPS, the server proves its identity to the client. With mutual TLS, both communicating sides authenticate each other using certificates.



10. Why mTLS?

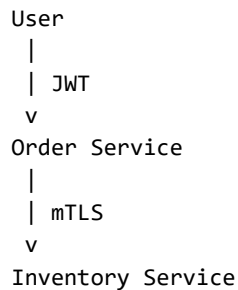
mTLS provides encryption plus mutual workload authentication for service-to-service communication.



11. mTLS Does Not Replace OAuth2

This distinction is essential:

mTLS → Which SERVICE is communicating?
JWT → Which USER is the request acting for?



These are complementary security layers.

12. Install Istio

```
istioctl version
kubectl get nodes
istioctl install --set profile=demo -y
kubectl get pods -n istio-system
```

13. Enable Automatic Sidecar Injection

```
kubectl label namespace ecommerce istio-injection=enabled
kubectl get namespace ecommerce --show-labels
```

```
kubectl rollout restart deployment -n ecommerce
kubectl get pods -n ecommerce
```

Existing Pods need to be recreated/restarted so that the injection mechanism can add the Envoy container.

14. Verify Sidecars

Workloads participating in the mesh should normally show 2/2 containers: the application and istio-proxy.

```
kubectl get pods -n ecommerce
```

Example:

```
product-service-xxxxx    2/2
order-service-xxxxx     2/2
inventory-service-xxxxx  2/2
```

15. Enforce STRICT mTLS

Create peer-authentication.yaml:

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: ecommerce
spec:
  mtls:
    mode: STRICT

kubectl apply -f peer-authentication.yaml
kubectl get peerauthentication -n ecommerce
```

STRICT means workloads covered by the policy must use mutual TLS; plaintext traffic is not accepted under that policy.

16. Verify mTLS

```
istioctl authn tls-check product-service.ecommerce.svc.cluster.local
```

The exact output can vary by Istio version. The goal is to confirm that the relevant service-to-service communication is using mTLS.

Part 3 — Traffic Management

17. Canary Deployment

Suppose Product Service has stable and canary versions and we want a weighted routing policy:

```
stable  ≈ 80%
canary  ≈ 20%
```

Unlike using replica counts as an approximation, Istio lets us express the desired routing weights directly.

18. VirtualService

VirtualService defines routing rules.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: product-service
  namespace: ecommerce
spec:
  hosts:
    - product-service
  http:
    - route:
        - destination:
            host: product-service
            subset: stable
            weight: 80
        - destination:
            host: product-service
            subset: canary
            weight: 20

kubectl apply -f virtual-service.yaml
```

19. DestinationRule

DestinationRule defines destination-level policies and subsets.

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: product-service
  namespace: ecommerce
spec:
  host: product-service
  subsets:
    - name: stable
      labels:
        version: stable
    - name: canary
      labels:
        version: canary

kubectl apply -f destination-rule.yaml
```

20. Kubernetes Labels Matter

The subset labels must match labels actually present on the Pods.

```
stable Pod:  
app=product-service  
version=stable
```

```
canary Pod:  
app=product-service  
version=canary
```

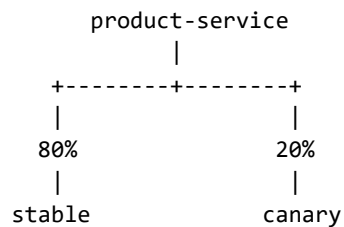
```
kubectl get pods -n ecommerce --show-labels
```

21. VirtualService vs DestinationRule

Remember:

```
VirtualService    → WHERE should traffic go?  
DestinationRule  → WHAT policy applies to the destination?
```

22. Weighted Traffic



The weights configure the routing policy. A small sample of requests will not necessarily be exactly 80/20; over a sufficiently large number of requests, the observed distribution should approach the configured weights.

Part 4 — Mesh-Level Resilience

23. Why Outlier Detection?

If one destination instance repeatedly fails, the mesh can detect it and temporarily eject that endpoint from the load-balancing pool.

Product Pod A	Healthy
Product Pod B	Healthy
Product Pod C	Failing

Repeated failures

|

v

Envoy detects outlier

|

v

Endpoint can be ejected

24. DestinationRule with Outlier Detection

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: product-service
  namespace: ecommerce
spec:
  host: product-service

  subsets:
    - name: stable
      labels:
        version: stable
    - name: canary
      labels:
        version: canary

  trafficPolicy:
    outlierDetection:
      consecutive5xxErrors: 5
      interval: 30s
      baseEjectionTime: 30s
      maxEjectionPercent: 50
```

```
kubectl apply -f destination-rule.yaml
```

25. Configuration Meaning

- `consecutive5xxErrors: 5` — five consecutive HTTP 5xx responses can trigger outlier handling.
- `interval: 30s` — the proxy evaluates outlier detection at the configured interval.
- `baseEjectionTime: 30s` — an ejected endpoint can be removed from the load-balancing pool for the configured period.
- `maxEjectionPercent: 50` — limits how much of the destination can be ejected by outlier detection.

26. Resilience4j vs Istio

Resilience4j operates inside the application. Istio operates at the mesh/proxy layer.

```

Resilience4j
Application
  +-- Retry
  +-- Timeout
  +-- Circuit Breaker
  +-- Business fallback

```

```

Istio
Application
  |
  Envoy
    +-- Traffic policy
    +-- Failure detection
    +-- Endpoint ejection
    +-- Routing

```

27. When to Use Resilience4j

- Business-aware fallback logic
- Domain-specific recovery
- Application-specific behavior
- Different behavior depending on the operation
- Responses such as OrderStatus.PENDING

28. When to Use Istio

- Uniform networking policies
- Language-independent resilience
- Mesh-wide traffic management
- mTLS
- Canary routing
- Endpoint ejection

29. Can We Use Both?

Yes. They operate at different layers.

```

Request
  |
  Istio Envoy
    | infrastructure-level resilience
    v
Spring Boot application
  |
  Resilience4j
    | business-aware fallback

```

30. Decision Matrix

Requirement	Best Fit
Business-aware fallback	Resilience4j
Return domain-specific response	Resilience4j
Mesh-wide traffic policy	Istio
mTLS between services	Istio
Language-independent resilience	Istio
Canary traffic splitting	Istio

Endpoint ejection	Istio
Defense in depth	Both

Part 5 — Testing and Troubleshooting

31. Verify Istio and Workloads

```
istioctl version
istioctl verify-install
kubectl get pods -n istio-system
kubectl get pods -n ecommerce
kubectl get peerauthentication -n ecommerce
kubectl get virtualservice -n ecommerce
kubectl get destinationrule -n ecommerce
```

32. Inspect Envoy

```
kubectl describe pod <pod-name> -n ecommerce
kubectl logs <pod-name> -c istio-proxy -n ecommerce
istioctl proxy-config cluster <pod-name> -n ecommerce
istioctl proxy-config listener <pod-name> -n ecommerce
istioctl proxy-config routes <pod-name> -n ecommerce
```

33. Pods Still Show 1/1

```
kubectl get namespace ecommerce --show-labels
kubectl label namespace ecommerce istio-injection=enabled
kubectl rollout restart deployment -n ecommerce
kubectl get pods -n ecommerce
```

34. Communication Breaks After STRICT mTLS

Check that the communicating workloads participate in the mesh and have the Envoy sidecar. STRICT mTLS rejects plaintext traffic where the policy applies.

```
kubectl get pods -n ecommerce
kubectl describe pod <pod-name> -n ecommerce
kubectl logs <pod-name> -c istio-proxy -n ecommerce
```

35. VirtualService Does Not Split Traffic

```
kubectl get virtualservice -n ecommerce
kubectl get destinationrule -n ecommerce
kubectl get pods -n ecommerce --show-labels
```

Verify that the Pod labels exactly match the subset selectors.

36. Testing Weighted Traffic

Generate many requests against the appropriate Product Service endpoint in your course environment.

```
Linux/macOS:
for i in {1..100}; do
  curl http://<endpoint>/products
done

Windows cmd:
for /L %i in (1,1,100) do curl http://<endpoint>/products
```

The observed traffic should trend toward the configured 80/20 policy over a sufficiently large sample.

Part 6 — Hands-On Lab

37. Lab Objective

- Install Istio.
- Enable automatic sidecar injection.
- Verify Envoy.
- Enable STRICT mTLS.
- Create stable and canary subsets.
- Configure 80/20 traffic routing.
- Configure outlier detection.
- Explain the role of Resilience4j versus Istio.

38. Lab Step 1 — Install Istio

```
istioctl install --set profile=demo -y
kubectl get pods -n istio-system
```

39. Lab Step 2 — Enable Injection

```
kubectl label namespace ecommerce istio-injection=enabled
kubectl rollout restart deployment -n ecommerce
kubectl get pods -n ecommerce
```

Expected: mesh workloads show 2/2 containers.

40. Lab Step 3 — Enable STRICT mTLS

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: ecommerce
spec:
  mtls:
    mode: STRICT

kubectl apply -f peer-authentication.yaml
kubectl get peerauthentication -n ecommerce
```

41. Lab Step 4 — Configure DestinationRule

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: product-service
  namespace: ecommerce
spec:
  host: product-service
  subsets:
    - name: stable
      labels:
        version: stable
    - name: canary
      labels:
        version: canary
  trafficPolicy:
```

```
outlierDetection:
  consecutive5xxErrors: 5
  interval: 30s
  baseEjectionTime: 30s
  maxEjectionPercent: 50
```

```
kubectl apply -f destination-rule.yaml
```

42. Lab Step 5 — Configure VirtualService

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: product-service
  namespace: ecommerce
spec:
  hosts:
    - product-service
  http:
    - route:
        - destination:
            host: product-service
            subset: stable
            weight: 80
        - destination:
            host: product-service
            subset: canary
            weight: 20
```

```
kubectl apply -f virtual-service.yaml
```

43. Lab Step 6 — Verify

```
kubectl get virtualservice -n ecommerce
kubectl get destinationrule -n ecommerce
kubectl get peerauthentication -n ecommerce
kubectl get pods -n ecommerce --show-labels
```

44. Lab Step 7 — Test Traffic

Generate a sufficiently large number of requests and observe which Product Service version responds. The goal is for the distribution to trend toward approximately 80% stable and 20% canary.

Part 7 — Architecture Exercise

45. Scenario

Your company has 30 microservices, five development teams, and services written in Java, Node.js, Go, and Python. Each team independently implements retries, timeouts, circuit breakers, tracing, authentication, and TLS.

Questions

- Would you recommend Istio? Explain why.
- Should the company remove Resilience4j completely? Why or why not?
- Which concerns belong in the mesh and which belong in the application?
- What operational costs would Istio introduce?
- What evidence would justify adding a service mesh?

46. Architectural Trade-Off

Istio is powerful, but it is not free. It adds sidecars, CPU and memory overhead, a control plane, configuration complexity, operational responsibility, and a learning curve.

A small platform with three Spring Boot services and one team may not justify the overhead. A larger multi-team, multi-language platform with strong security and traffic-management requirements may benefit much more.

Never justify Istio simply by saying: 'It is an enterprise technology.' Justify it by identifying a real architectural problem it solves.

Part 8 — Final Mental Model

47. Four Concepts to Remember

1. Sidecar
Application + Envoy
2. Data Plane
Envoy handles actual traffic
3. Control Plane
Istiod manages and distributes configuration
4. Service Mesh
A network infrastructure layer around service-to-service communication

48. Security Mental Model

mTLS → Which SERVICE?
JWT → Which USER?

```
graph TD
    User -->|JWT| OrderService[Order Service]
    OrderService -->|mTLS| InventoryService[Inventory Service]
```

49. Traffic Mental Model

```
graph TD
    VirtualService -->|WHERE should traffic go?| DestinationRule
    DestinationRule -->|WHAT policy applies to the destination?|
```

50. Resilience Mental Model

```
graph TD
    Istio --> Infra[Infrastructure/network resilience]
    Resilience4j --> App[Application/business resilience]
```

51. Final Knowledge Check

Q1. What is a sidecar proxy?

Answer: A second container in the same Pod as the application that can intercept and manage network traffic.

Q2. What is the Data Plane?

Answer: The Envoy proxies that handle actual network traffic.

Q3. What is the Control Plane?

Answer: Istiod, which distributes configuration and policies to the proxies.

Q4. Why does Istio not require Spring Boot changes?

Answer: The networking behavior is implemented by the proxy/infrastructure layer.

Q5. What does mTLS provide?

Answer: Encrypted communication and mutual authentication between workloads.

Q6. mTLS versus OAuth2?

Answer: mTLS authenticates the communicating service; OAuth2/JWT represents the user/client identity and authorization context.

Q7. What is a VirtualService?

Answer: A resource for traffic-routing rules.

Q8. What is a DestinationRule?

Answer: A resource for destination policies and subsets.

Q9. What does weight: 80 mean?

Answer: An 80 routing weight relative to the other routes in that routing rule.

Q10. What does outlier detection do?

Answer: It can detect failing destination instances and temporarily eject them from the load-balancing pool.

Q11. Can Istio and Resilience4j be used together?

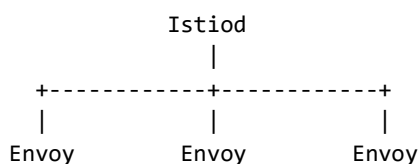
Answer: Yes. They operate at different layers and can provide complementary resilience.

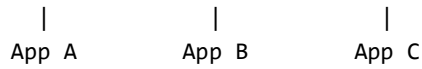
Q12. Why should we not automatically add Istio?

Answer: Because it adds operational complexity, resource overhead, and a learning curve that must be justified by real requirements.

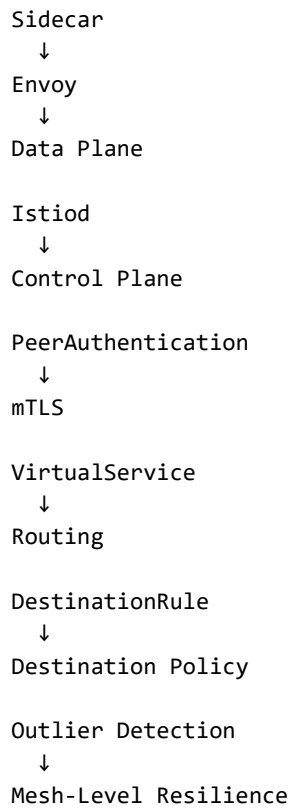
52. Session Summary

A service mesh moves many cross-cutting networking concerns from application code into infrastructure. Istio does this primarily through Envoy sidecar proxies, while Istiod manages the control-plane configuration.





The core ideas are:



The final architectural rule is simple: business-aware behavior belongs in the application; infrastructure/network behavior can belong in the mesh. Resilience4j and Istio can therefore complement each other.

53. Scope Boundary

This session teaches how to understand and use Istio at the architecture and application-platform level. It does not attempt to teach Envoy internals or replace the Kubernetes knowledge covered in earlier sessions. Envoy is treated as the proxy managed by Istio rather than as a separate low-level system to configure manually.